KOENIG

step forward



Stateless Services

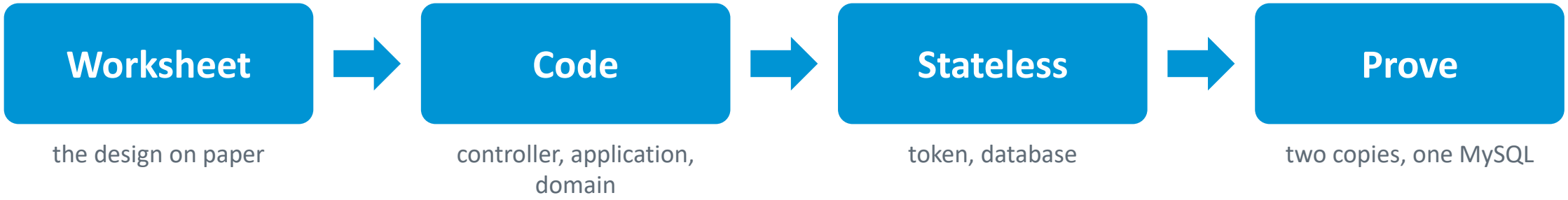
From a service worksheet to a running Kotlin Spring Boot service

What You Will Learn

- What stateless means and why
- Where the state goes
- From worksheet to a stored aggregate
- Use cases as REST resources
- Identity: a token, no session
- Run two copies and prove it

Demo Scenario: Learnly Enrollment Must Scale

The design gave Learnly an Enrollment service on paper. Exam week is coming and one copy will not be enough. We build it as its own Kotlin app with its own MySQL schema, so that any number of copies can run.



What can go wrong

- **Session in memory**
- **Last seat sold twice**
- **Double submit**

Where We Left the Design: the Worksheet

Worksheet item	Enrollment service
Purpose	Decides who may take which course
Responsibilities	Enroll a student, enforce seat limits, cancel an enrollment, list a student's courses (waiting list: optional challenge)
Does not do	Course prices, money, lesson progress
Owns (entities)	Enrollment (id, studentId, courseId, status) and SeatBatch (seats total, seats taken); value object EnrollmentStatus; aggregate root Enrollment; stored in its own MySQL schema
Asks	Course Catalog: is the course open, what does it cost? Payment: charge the student
Events	EnrollmentConfirmed, EnrollmentCancelled
Application service	EnrollmentApplicationService: enroll, cancel, list

This one page is the input of everything we build

What Is State?

Login session

The server remembers that s-77 signed in.

Counter in a field

A variable holds how many seats are left.

Half-filled wizard

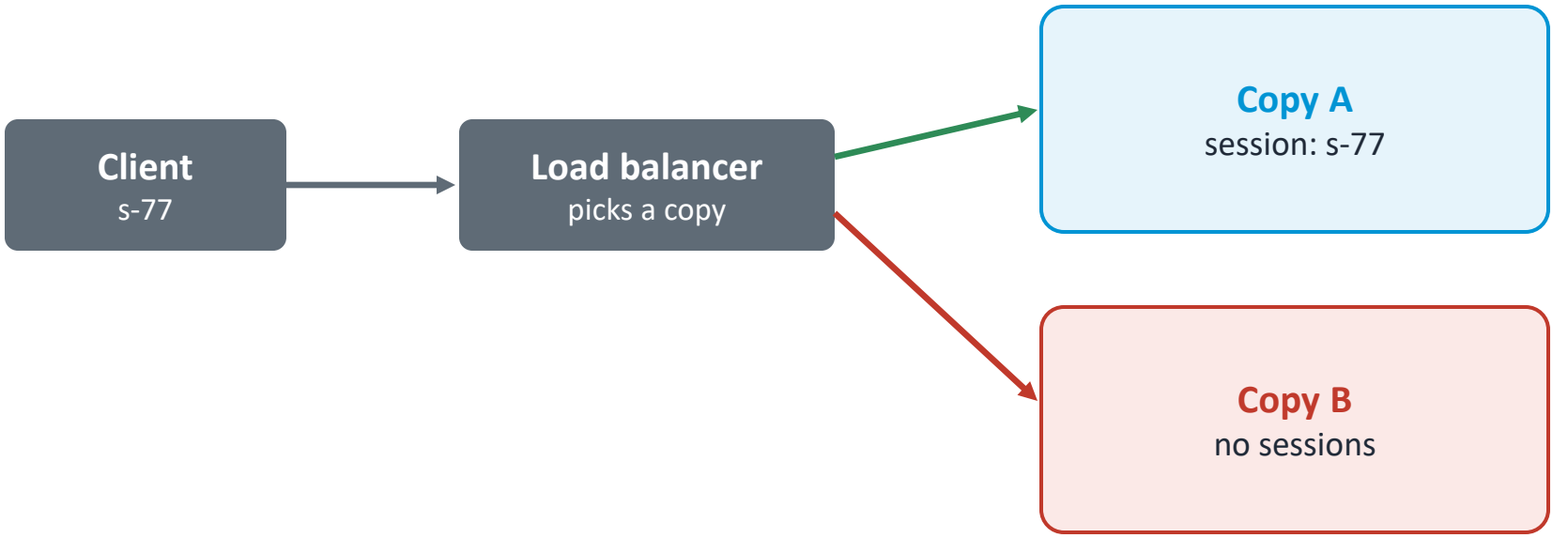
The server keeps step 2 of the enrollment form.

File on the local disk

An uploaded certificate stays on this machine.

State is anything a server remembers between two requests

What a Stateful Server Breaks

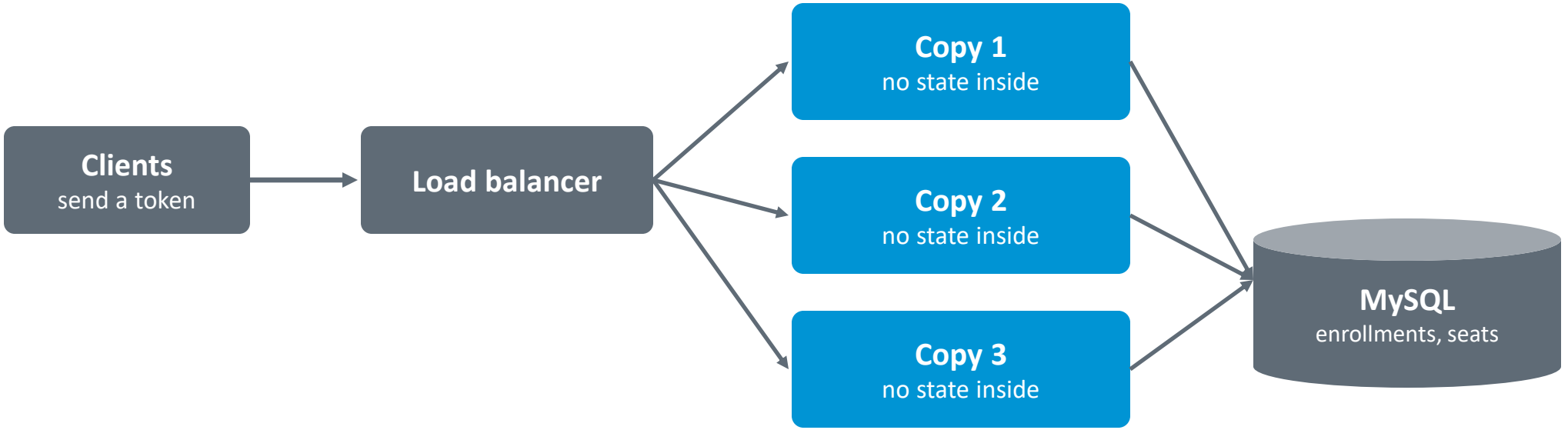


1. login goes to A
2. next request goes to B
3. B does not know the student

Result: the student is logged out at random, and the same question gets different answers.

A request that lands on another copy finds nothing

Stateless Server, Stateful System



Copies are cheap: add or remove them at any time. The database is the one place that must be looked after carefully.

The system keeps its state; the copies do not

Why Scaling Needs It

Scale out

Add copies in exam week, remove them afterwards.

Replace any copy

A crashed copy loses nothing; start a new one.

Deploy without logout

Roll out a new version one copy at a time.

If any copy can answer any request, copies become cheap

Stateful and Stateless Compared

Question	Stateful server	Stateless server
Who are you?	A session in the server's memory	A token that comes with every request
Where is the seat count?	A field in the code	A table in the database
Which copy can answer?	Only the one that remembers	Any copy
A copy crashes	Its users lose their session	Nothing is lost
Adding a copy	Needs sticky routing or copying of sessions	Start it and it works
Cost	Easy to code, hard to scale	Every request carries a little more

Stateless is the default for services that must scale

How We Build the Service

Read the worksheet

what to build, what not to build

Store the aggregate

entities, repositories, the seat rule

Expose use cases

REST resources and status codes

Know who is calling

a signed token, no session

Run two copies

one MySQL schema, prove it

Every part ends with a live demo
in the same project

From the worksheet to a running service, one small stage at a time

Where Does the State Go?

Place	Holds	Learnly Enrollment example
The request	What this one call needs	courseId, page, size, a token
The token	Who is calling, role, expiry	student s-77, role STUDENT
The database	Data of record, in the MySQL schema of the service	enrollments, seat batches, events
A cache	A copy that can be rebuilt (later modules)	course titles from Course Catalog
The environment	Settings of this copy	port, secret, instance name

Five places hold state in a stateless service, none of them is the copy

Shared State Needs One Owner

Where the value lives	With two copies	Verdict
A field in the code: seatsTaken	Each copy counts only its own requests	Wrong: seats are oversold
A local file	The other copy cannot see it	Wrong
A table owned by Enrollment	Both copies read and write the same row	Right: one truth
A table of another service	You would break the rule of the design	Wrong: ask by id instead
One schema per service	Own Spring Boot app, own MySQL schema	Right: database per service

If two copies could disagree, the value does not belong in the process

From Worksheet to Packages

Worksheet says	Code lives in
Entities, aggregate root	domain: Enrollment, SeatBatch
Value object	domain: EnrollmentStatus
Owned data access	domain: EnrollmentRepository, SeatBatchRepository
Application service	application: EnrollmentApplicationService
Asks	application: CourseCatalogPort, PaymentPort; stub: fixed answers
Events	domain: OutboxEvent, OutboxRepository
The API	controller: EnrollmentController; dto: EnrollmentRequest, EnrollmentResponse
Errors	exception: BusinessException, ApiExceptionHandler
Settings and seed data	config: AppProperties, WebConfig, SeatBatchSeeder
Who is calling	security: TokenService, AuthInterceptor

controller -> application service -> aggregate -> repository

Controller, DTO, Service: Who Does What

Controller
package: controller

- reads the request DTO
- calls ONE application service
- returns a response DTO

→

Application service
package: application

- one method per use case
- transaction
- throws BusinessException

→

Aggregate + repository
package: domain

- the aggregate holds the rules
- the repository loads and saves
- knows nothing about HTTP

dto: EnrollmentRequest, EnrollmentResponse, PageResponse and toResponse() | **exception:** BusinessException; only ApiExceptionHandler knows HTTP

Rule: a controller never uses a repository, and a DTO is never written inside a controller.

Three small jobs, each in its own package

The Aggregate Root and the Value Object

- **Root:** Enrollment is how the outside reaches the aggregate.
- **By id:** studentId and courseId are plain strings; the data lives elsewhere.
- **Value object:** EnrollmentStatus has two values only.
- **Behaviour:** cancel() knows it can be repeated; no free setters.

```
enum class EnrollmentStatus {
    CONFIRMED, CANCELLED
}
@Entity
class Enrollment(
    val studentId: String,  // Identity
    val courseId: String,   // Catalog
) {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null
    @Enumerated(EnumType.STRING)
    var status = EnrollmentStatus.CONFIRMED
        private set

    fun cancel() {
        status = EnrollmentStatus.CANCELLED
    }
}
```

Other contexts are referenced by id, never by object

The Repository and the Seat Rule

- **SeatBatch:** seatsTotal and seatsTaken, owned by Enrollment.
- **takeSeat:** adds one only while seatsTaken < seatsTotal.
- **Returns 0 or 1:** 0 means the course is full.
- **releaseSeat:** gives one back, never below zero.

```
interface SeatBatchRepository :
      JpaRepository<SeatBatch, String> {

   @Modifying
   @Query("update SeatBatch b set b.seatsTaken = b.seatsTaken + 1 " +
      "where b.courseId = :id " +
      "and b.seatsTaken < b.seatsTotal")
   fun takeSeat(@Param("id") courseId: String): Int
}
```

The rule is written as SQL: one UPDATE, not application code

The Last Seat: Let the Database Decide

Read, check, write in code

Copy 1 reads taken = 29 of 30
Copy 2 reads taken = 29 of 30
Both think a seat is free
Both write 30
Two students, one seat

One conditional UPDATE

update ... set seatsTaken = seatsTaken + 1
where seatsTaken < seatsTotal
The database runs them one after the other
The second one changes 0 rows
It is told: full

Rule: when a rule must hold across copies, put it in one atomic statement in the database.

Read, check, write in code is a race; one UPDATE is not

Demo: From Worksheet to a Stored Aggregate

What we do

- **Add:** JPA and the MySQL schema of the service.
- **Model:** Enrollment, EnrollmentStatus, SeatBatch.
- **Seed:** seat batches; read seats over REST.

What it shows

- **Mapping:** each worksheet line has a class.
- **Restart:** the numbers are still there.
- **Rule:** the seat limit is data.

Use Cases Become Methods

Use case (worksheet)	Method in EnrollmentApplicationService	REST resource
Enroll a student	enroll(student, course, key)	POST /enrollments
Cancel an enrollment	cancel(id, caller)	DELETE /enrollments/{id}
List a student's courses	listOf(student, page)	GET /enrollments?page=&size=
Read one enrollment	findFor(id, caller)	GET /enrollments/{id}

One method, one use case: the method coordinates; each aggregate protects its own rules.

The worksheet's application service lists the methods; the API exposes them

One Transaction Changes One Aggregate

- **Seat:** taking a seat changes SeatBatch, one small transaction.
- **Enrollment:** saving the enrollment and its event is a second small transaction.
- **If it fails:** give the seat back. This undo is called compensation.
- **Errors:** business exceptions with an ErrorKind, never an HTTP status.

```
fun enroll(student: String, course: String): Enrollment {
    if (seats.takeSeat(course) == 0)   // first small step
        throw BusinessException(ErrorKind.CONFLICT,
            "No seats left")
    if (!payment.charge(student, price)) {
        seats.releaseSeat(course)      // undo
        throw BusinessException(ErrorKind.CONFLICT,
            "Payment failed")
    }
    return store.saveEnrollment(student, course)  // second small step
}
```

Two aggregates mean two small steps; if the second fails, undo the first

Resources and URLs

Verb	URL	Meaning in Learnly Enrollment
POST	/enrollments	enroll the caller in a course
GET	/enrollments/{id}	read one enrollment
GET	/enrollments?page=0&size=10	the caller's own enrollments, one page
DELETE	/enrollments/{id}	cancel an enrollment
GET	/seat-batches/{courseId}	seats of a course: total, taken, left

Nouns in the path, the operation in the verb

HTTP Verbs Are the Contract

Verb	Use	Safe	Idempotent	Learnly
GET	Read	yes	yes	read an enrollment
POST	Create	no	no	enroll
PUT	Replace	no	yes	change a whole resource
DELETE	Remove	no	yes	cancel an enrollment

The verb tells everyone, including proxies and retries, what is safe

Status Codes That Tell the Story

Code	Meaning	Learnly Enrollment example
200 OK	Success with a body	GET an enrollment
201 Created	A new resource, with a Location header	POST /enrollments
204 No Content	Done, nothing to return	DELETE /enrollments/2
400 Bad Request	The request is wrong	blank courseId
404 Not Found	No such resource	unknown enrollment or course
409 Conflict	Clashes with the current state	already enrolled, course full

The code says what happened; the body says why

Errors as Problem Details

- **Business code:** throws BusinessException with an ErrorKind, never a status.
- **One handler:** ApiExceptionHandler maps ErrorKind to HTTP.
- **Same fields:** title, status, detail, instance.
- **Client logic:** read the status, show the detail.

```
// in the application service
throw BusinessException(
    ErrorKind.CONFLICT, "No seats left",
    "Course c-999 is full")

// in ApiExceptionHandler
ErrorKind.CONFLICT -> HttpStatus.CONFLICT

// the answer: 409 application/problem+json
{"title": "No seats left", "status": 409,
 "detail": "Course c-999 is full"}
```

Business code says what is wrong; one handler chooses the status

Paging in the Request

page 0
size 2

page 1
size 2

page 2
size 2

```
GET /enrollments?page=1&size=2

{"items":[...],"page":1,"size":2,"total":5}
```

State lives in the request:
page and size travel with every call.
Any copy can serve any page.

The server never remembers which page you saw

Demo: Use Cases as REST Resources

What we do

- **Build:** the application service, DTOs and the controller.
- **Answer:** 201 with Location, 204, 404, 409, 400.
- **Page:** list the caller's enrollments.

What it shows

- **Mapping:** worksheet use case to method to URL.
- **Errors:** one shape for all.
- **Seat rule:** decided by the database.

No Session: Send a Token

Session: the server remembers you

Client sends a cookie: session id
Only the copy that created it knows it
Other copies reject the request
Needs sticky routing or shared sessions

Token: you carry your identity

Client sends: Authorization: Bearer <token>
The token says who and until when
Any copy can check it
No session table at all

Signed token: any copy can check it with the shared secret; no call to Identity, no session table.

Identity travels in every request instead of living in the server

401 or 403

Status	Question	Learnly Enrollment
401 Unauthorized	Who are you?	no token, a bad token or an expired token
403 Forbidden	I know you, but may you?	s-88 reads the enrollment of s-77; an administrator may

Unknown caller or forbidden caller: two different answers

Demo: A Signed Token, No Session

What we do

- **Copy:** the token classes from the finished project; just use them.
- **Check:** an interceptor verifies it on every call.
- **Own:** the caller comes from the token; 401 and 403.

What it shows

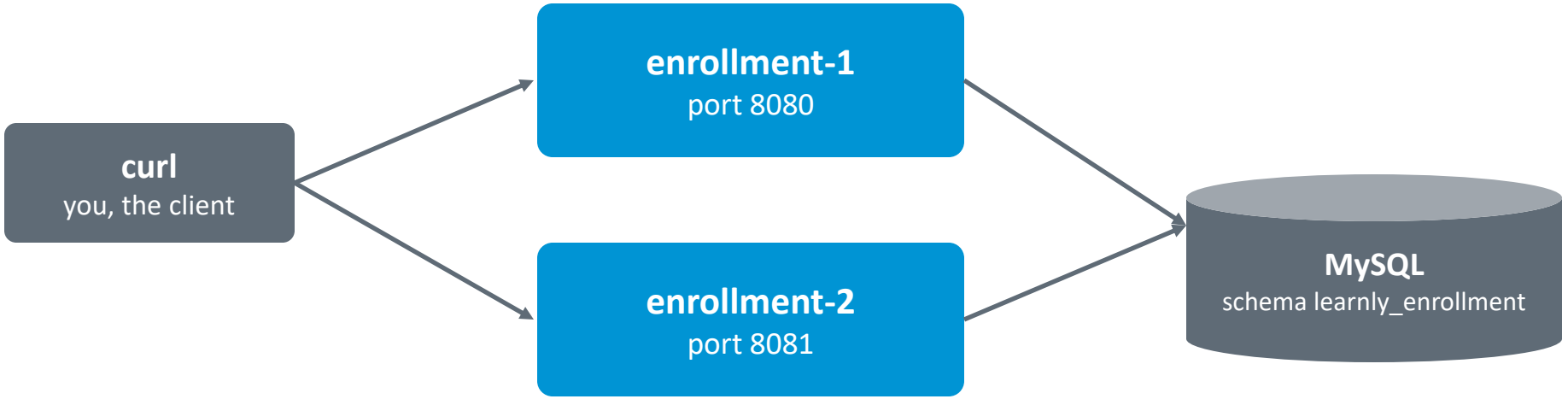
- **No session:** nothing stored about the login.
- **Any copy:** same check everywhere.
- **Safe:** nobody can enroll somebody else.

Configuration from the Environment

Source (strongest first)	Example	Use
Command line	--server.port=8081	one-off override
Environment variable	PORT=8081, INSTANCE_NAME=enrollment-2	per copy, per environment
application.properties	server.port=${PORT:8080}	defaults

The same jar becomes enrollment-1 or enrollment-2 by its environment

Two Copies, One Database



One service, one schema: the other Learnly services get their own app and their own schema, never this one.

Same jar, two ports, one shared MySQL schema

Demo: Configuration from the Environment

What we do

- **Start:** the same jar with another name.
- **Next:** run two copies.

What it shows

- **One jar:** two identities.
- **No code change:** only the environment differs.

Key Takeaways

Stateless
No memory in the copy: any copy answers.

From worksheet to service
Read it, store it, expose it, protect it, prove it.

State goes
Request, token, database, environment.

Safe calls
Token for identity, one UPDATE for the last seat.

Proof
Two copies, one MySQL schema.



Happy Learning :)